

Customer Support Intelligence & Ticket Automation System

Updated Industry-Ready NLP Roadmap — aligned with the primary 100K IT Support Tickets dataset

1. Project Goal

Build an end-to-end customer-support intelligence platform that takes incoming support tickets and automatically understands them, predicts the issue type, analyses customer sentiment, predicts ticket priority, retrieves similar historical tickets, identifies relevant resolutions, and recommends a response for a human support agent.

Main principle: this is not just a notebook or a chatbot. The final project should demonstrate NLP modelling, Transformer models, semantic search, hybrid text + structured-data modelling, APIs, databases, a dashboard, evaluation, explainability and production-style engineering.

2. Primary Dataset

IT Support Tickets — 100K records

The selected primary dataset contains approximately 100,000 synthetic SaaS/IT support tickets with structured metadata and support text. The dataset is suitable for the core project because it contains fields such as ticket ID, timestamp, customer ID, customer segment, channel, product area, issue type, priority, status, SLA plan, initial customer message, agent first reply, resolution summary, resolution time, reopened status, customer sentiment, CSAT score, attachment flag, platform and region.

Important limitation: the dataset is synthetic. Therefore, project results should be presented as experimental results on this dataset and not as proof of performance on live production customer data.

3. Dataset-to-Project Mapping

Dataset field	How it will be used
initial_message	Main NLP input for intent/issue classification, sentiment and semantic retrieval
issue_type	Primary supervised target for ticket classification
priority	Primary target for priority prediction
customer_sentiment	Supervised target for sentiment analysis
agent_first_reply	Historical support response for response recommendation experiments
resolution_summary	Resolution knowledge for retrieval/RAG-style response recommendations
customer_id	Customer-history analysis; use carefully to avoid leakage
customer_segment	Structured feature for hybrid priority modelling
channel	Structured feature for hybrid priority modelling
product_area	Structured feature and useful context for classification
sla_plan	Useful structured feature for priority/urgency modelling
status	Operational analytics; avoid leakage if it is determined after resolution
resolution_time_hours	Resolution-time analysis/prediction extension
reopened	Operational analysis / future risk feature
csat_score	Customer satisfaction analysis; potential downstream analysis
platform / region	Structured context and analytics
created_at	Temporal analysis; possible time-based validation

4. Final System Architecture

Incoming Ticket → Validation/Cleaning → NLP Models → Structured Feature Processing → Predictions → Sentence-BERT Embedding → Vector Search → Relevant Historical Tickets/Resolutions → Response Recommendation → Human Agent → Database/Feedback → Analytics

The system should contain these main components:

- **Intent/Issue Classifier:** predicts the type of support problem.
- **Sentiment Model:** predicts positive, neutral or negative customer sentiment.
- **Priority Model:** predicts low/medium/high/critical using text and selected structured features.
- **Semantic Retrieval:** finds similar historical tickets using embeddings.
- **Resolution Retrieval:** retrieves relevant historical resolutions and agent responses.
- **Response Recommendation:** proposes a response grounded in retrieved information.
- **Agent Dashboard:** lets a human review predictions and accept, edit or reject recommendations.
- **API:** serves the models through FastAPI.
- **Database:** stores tickets, predictions, retrieval results and agent feedback.

5. Phase 0 — Project Setup

- Create a GitHub repository with a clear README.
- Create a Python virtual environment and requirements file.
- Set up folders for data, notebooks, source code, API, dashboard, models and tests.
- Create a configuration file for paths and model settings.
- Set random seeds where appropriate for reproducibility.
- Create a project log documenting major decisions, experiments and results.

Suggested repository

```
customer-support-ai/  
  data/raw/  
  data/processed/  
  notebooks/  
  src/  
    preprocessing/  
    features/  
    models/  
    retrieval/  
    evaluation/  
  api/  
  app/  
  models/  
  tests/  
  configs/  
  Dockerfile  
  docker-compose.yml  
  requirements.txt  
  README.md
```

6. Phase 1 — Dataset Acquisition and Inspection

- Download the primary 100K IT Support Tickets dataset and preserve the original raw file unchanged.
- Load the dataset with pandas and inspect shape, column names, data types and sample records.
- Check the number of unique values for issue_type, priority and customer_sentiment.
- Measure missing values, duplicates and suspicious records.

- Inspect ticket text length, word counts and empty messages.
- Analyse class distributions before deciding on resampling or class weights.
- Check whether agent responses and resolutions contain information that would leak the target during training.

Deliverables

EDA notebook + data dictionary + class-distribution charts + missing-value report + duplicate report + documented leakage risks.

7. Phase 2 — Data Cleaning and Leakage Control

- Remove exact duplicate tickets where appropriate.
- Normalise missing/unknown values.
- Clean text conservatively: whitespace, malformed characters and obvious formatting noise.
- Do not aggressively remove stop words or punctuation before Transformer modelling.
- Create separate text-processing functions for classical ML and Transformer models.
- Split data into train, validation and test sets before fitting vocabulary, TF-IDF or other learned preprocessing.
- Investigate near-duplicates and repeated customer conversations.
- Do not use post-resolution fields such as final status or resolution time as predictors if they would not be available when the ticket arrives.

8. Phase 3 — Define the Prediction Tasks

Task	Input	Target	Main purpose
Issue classification	initial_message + optional context	issue_type	Understand what the customer needs
Sentiment analysis	initial_message	customer_sentiment	Detect customer emotional state
Priority prediction	message + available ticket metadata	priority	Identify urgency
Semantic retrieval	initial_message	similar historical tickets	Find relevant cases
Response recommendation	new ticket + retrieved evidence	recommended response	Assist human agents

9. Phase 4 — Build Classical NLP Baselines

Before using Transformers, establish strong and understandable baselines. This makes the final model comparison credible and shows that the project is based on experimentation rather than simply using a pretrained model.

- Convert `initial_message` into TF-IDF features using a fitted training-set vectorizer.
- Train Logistic Regression for `issue_type`.
- Train Linear SVM and compare it with Logistic Regression.
- Use class weights or other justified imbalance handling where needed.
- Record training time, inference time and all evaluation metrics.
- Save the best baseline model and preprocessing object.

Baseline evaluation

Use Accuracy, Macro Precision, Macro Recall, Macro-F1, per-class F1 and a confusion matrix. Do not report only accuracy when classes are imbalanced.

10. Phase 5 — Transformer-Based Issue Classification

- Use Hugging Face Transformers with a model such as DistilBERT/BERT.
- Tokenise `initial_message` with truncation and an appropriate maximum sequence length.
- Fine-tune the classifier using the training set and validate during training.
- Use early stopping/model checkpointing where appropriate.
- Compare the Transformer against TF-IDF + Logistic Regression and TF-IDF + SVM.
- Measure model size and inference latency as well as predictive quality.

Expected experiment table

Model	Accuracy	Macro Precision	Macro Recall	Macro-F1	Inference Time
TF-IDF + Logistic Regression	actual result	actual result	actual result	actual result	actual result
TF-IDF + Linear SVM	actual result	actual result	actual result	actual result	actual result
DistilBERT/BERT	actual result	actual result	actual result	actual result	actual result

11. Phase 6 — Sentiment Analysis

- Use `customer_sentiment` as the supervised target.
- Start with a simple baseline and then evaluate a Transformer-based classifier.
- Inspect whether sentiment is strongly imbalanced.
- Perform per-class analysis and inspect incorrect examples.
- Create a confidence score for the dashboard.
- Do not assume sentiment is perfectly objective; document dataset-label limitations.

12. Phase 7 — Priority Prediction: Hybrid NLP + Structured Model

This is a key upgrade to make the project stronger. Priority should not be treated only as a text-classification problem. Build a hybrid model that combines text representation with structured information available when the ticket arrives.

Text branch: `initial_message` → Transformer/TF-IDF embedding
Structured branch: `customer_segment` + `channel` + `product_area` + `sla_plan` + other valid pre-resolution features
Combined representation → priority classifier

- First build a structured-only baseline such as Logistic Regression, Random Forest or XGBoost.
- Build a text-only priority model.

- Build the combined hybrid model.
- Compare all three approaches.
- Pay special attention to recall for high/critical priority tickets.
- Exclude fields that become known only after the ticket is resolved.

13. Phase 8 — Sentence-BERT Semantic Retrieval

- Create embeddings for historical initial_message text using Sentence-BERT.
- Store vectors in FAISS initially.
- For a new ticket, generate its embedding and search the vector index.
- Return top-K similar historical tickets.
- Display similarity scores and ticket metadata.
- Do not use the test set to construct retrieval knowledge available during training/evaluation.

Retrieval evaluation

Use a manually evaluated sample or a carefully defined relevance benchmark. Measure Precision@K, Recall@K and MRR where ground-truth relevance can be established. Also perform qualitative inspection of retrieved cases.

14. Phase 9 — Resolution and Response Recommendation

Use agent_first_reply and resolution_summary as historical support knowledge. The goal is not to copy a response blindly. The system should retrieve relevant evidence and use it to assist an agent.

New ticket → retrieve similar tickets → retrieve relevant resolutions/replies → construct grounded context → generate or recommend response → human agent review

- Start with a retrieval-only recommendation system.
- Then experiment with a controlled RAG workflow if an appropriate LLM is available.
- Clearly separate retrieved facts from generated wording.
- Prevent unsupported claims by requiring the response to be grounded in retrieved resolution information.
- Show the source tickets used for the recommendation in the dashboard.
- Keep Accept/Edit/Reject controls so a human remains responsible for the final response.

15. Phase 10 — Database Design

Table	Purpose
tickets	Original ticket and operational metadata
predictions	Intent, sentiment, priority and confidence
similar_tickets	Retrieved ticket IDs and similarity scores
recommendations	Suggested response and supporting evidence
agent_feedback	Accept/edit/reject actions and edited response
model_versions	Model name, version and deployment information

16. Phase 11 — FastAPI Backend

- Create a /health endpoint for deployment checks.
- Create POST /predict for intent, sentiment and priority predictions.
- Create GET /similar-tickets for semantic retrieval.
- Create POST /generate-response for response recommendation.
- Create GET /tickets/{id} for ticket details.
- Create GET /analytics for dashboard statistics.
- Validate input data and return clear error messages.
- Log model version, request ID and latency without storing unnecessary sensitive information.

17. Phase 12 — Streamlit Agent Dashboard

The dashboard should look like an internal support-agent tool rather than a model demo.

```
Ticket text box
↓
[ Analyse Ticket ]
↓
Category | Sentiment | Priority | Confidence
↓
Similar historical tickets
↓
Relevant resolutions / supporting evidence
↓
Suggested response
↓
[ Accept ] [ Edit ] [ Reject ]
```

- Add a ticket search/history page.

- Add a model-confidence view.
- Add similar-ticket evidence.
- Add response editing and feedback.
- Add operational analytics such as ticket volume, category, priority and sentiment.

18. Phase 13 — Explainability

- For TF-IDF models, use SHAP or feature-weight explanations to show influential terms.
- For structured priority models, show important features.
- For Transformer models, use an appropriate attribution method cautiously.
- Display explanations as supporting evidence, not as proof of causal relationships.
- Add an error-analysis notebook showing representative false positives and false negatives.

19. Phase 14 — Evaluation Framework

Component	Metrics / checks
Issue classification	Accuracy, Macro Precision, Macro Recall, Macro-F1, per-class F1, confusion matrix
Sentiment	Precision, Recall, F1, confusion matrix, per-class performance
Priority	Macro-F1, High/Critical Recall, confusion matrix, calibration/confidence analysis
Retrieval	Precision@K, Recall@K, MRR, qualitative relevance review
Response recommendation	Relevance, groundedness/faithfulness, human review
System	Latency, p95 latency, error rate, throughput

20. Phase 15 — Robustness and Generalisation

Use the second customer-support dataset discussed earlier as an optional independent validation/robustness dataset. Because its available fields do not perfectly match the primary dataset, do not force every model to run unchanged. Instead, use it for compatible tasks such as text classification and qualitative/generalisation analysis, with the differences documented clearly.

- Map only genuinely equivalent categories/labels.
- Do not invent missing sentiment or priority labels.
- Report domain differences and label differences.
- Treat cross-dataset testing as a robustness experiment, not as proof of universal generalisation.

21. Phase 16 — Testing

- Unit-test preprocessing functions.
- Test API input validation and error handling.
- Test prediction output schema.
- Test retrieval for known example tickets.
- Test that model and tokenizer versions are loaded consistently.
- Add a small regression-test dataset so future changes can be checked.

22. Phase 17 — Docker and Deployment

- Create a Dockerfile for the application.
- Use Docker Compose locally if running API, database and vector search as separate services.
- Keep model files and secrets outside the source code where appropriate.
- Deploy the API and dashboard to a suitable cloud platform.
- Add a health-check endpoint and deployment documentation.
- Record system latency after deployment.

23. Phase 18 — Logging and Monitoring

Track request ID, timestamp, endpoint, model version, prediction, confidence, latency and error type. Avoid logging unnecessary customer information. Add monitoring for API failures and latency.

24. Final End-to-End Workflow

Customer Ticket → Input Validation → Text/Metadata Processing → Issue Classification + Sentiment + Hybrid Priority Model → Sentence-BERT Embedding → FAISS Retrieval → Historical Resolution Retrieval → Grounded Response Recommendation → Human Agent → Feedback Storage → Analytics/Monitoring

25. Recommended Experiment Plan

Experiment	Purpose
E1: TF-IDF + Logistic Regression	Simple issue-classification baseline
E2: TF-IDF + Linear SVM	Stronger classical baseline
E3: DistilBERT/BERT	Transformer comparison
E4: Sentiment baseline vs Transformer	Measure sentiment improvement
E5: Structured-only priority model	Measure value of metadata
E6: Text-only priority model	Measure value of ticket text
E7: Hybrid priority model	Measure combined text + metadata value
E8: Sentence-BERT retrieval	Build semantic search
E9: Retrieval-based response recommendation	Create first response-assist system
E10: RAG response recommendation	Test grounded generation

26. Suggested Timeline

Week	Main work
1	Environment, repository, dataset acquisition and EDA
2	Cleaning, leakage analysis, splitting and data pipeline
3	TF-IDF baselines and evaluation

Week	Main work
4	Transformer issue classifier
5	Sentiment model + error analysis
6	Priority models: structured, text-only and hybrid
7	Sentence-BERT + FAISS retrieval
8	Resolution retrieval + response recommendation
9	FastAPI + PostgreSQL
10	Streamlit agent dashboard
11	Explainability, testing and robustness
12	Docker, deployment, monitoring, README and final polishing

27. What the Final GitHub Project Should Demonstrate

- Strong Python and data-engineering practices.
- Classical NLP and Transformer-based NLP.
- Multi-task support intelligence: issue, sentiment and priority.
- Hybrid modelling using unstructured text plus structured ticket metadata.
- Semantic search using Sentence-BERT and FAISS.
- Grounded response recommendation/RAG.
- Proper evaluation rather than accuracy-only reporting.
- Explainability and error analysis.
- FastAPI, PostgreSQL and Streamlit integration.
- Docker, testing, logging and deployment.

28. Resume-Ready Project Description

Customer Support Intelligence & Ticket Automation Platform

Python, PyTorch, Hugging Face Transformers, Sentence-BERT, FAISS, FastAPI, PostgreSQL, Streamlit, Docker

- Developed an end-to-end NLP platform for support-ticket issue classification, sentiment analysis, priority prediction and semantic retrieval.
- Compared TF-IDF/Logistic Regression, Linear SVM and Transformer-based classifiers using Macro-F1, precision, recall and per-class evaluation.
- Designed a hybrid priority model combining ticket-text representations with structured customer, channel, product and SLA features.
- Implemented Sentence-BERT + FAISS semantic retrieval to identify relevant historical tickets and resolutions.
- Built a grounded response-recommendation workflow and human-in-the-loop agent dashboard using FastAPI, PostgreSQL and Streamlit.

Important: replace any performance claims in the resume with the actual values obtained from your experiments.

29. MVP vs Full Resume-Level Version

Feature	MVP	Full version
EDA + cleaning	✓	✓
TF-IDF baseline	✓	✓
Transformer classifier	—	✓

Feature	MVP	Full version
Sentiment	—	✓
Priority prediction	Basic	Hybrid text + structured
Semantic search	—	✓
Resolution retrieval	—	✓
RAG response recommendation	—	✓
FastAPI	—	✓
PostgreSQL	—	✓
Streamlit	✓	✓
Explainability	—	✓
Testing	Basic	✓
Docker	—	✓
Deployment/monitoring	—	✓

30. Final Implementation Rule

Build the project in layers and record real results at every stage. Do not fabricate labels, metrics or business impact. The primary dataset is synthetic, so clearly state that limitation. The strongest final project is the one where every major component can be demonstrated live: a ticket goes in, the NLP models analyse it, similar cases are retrieved, evidence is shown, a response is recommended, an agent can edit/approve it, and the event is stored and monitored.